❷ Waits for the future to complete and extracts a value from Success

NOTE `ScalaFutures` works by being in scope and implicitly wrapping each future in an `org.scalatest.concurrent.FutureConcept`. This wrapper has the `futureValue` method, which gives you direct access to the completed successful `Future` in this example.

14.4.1 Mapping by column

To be able to refer to the individual columns of a CSV file, you need headers. Depending on whether the CSV file has them, you can use `CsvToMap.toMap()` or `CsvToMap.withHeaders()`. Both methods create a `Map [String, ByteString]` for each line, so the final result is `Seq[Map[String, ByteString]]`. The following listing shows an example of a CSV file that initially has no headers and how to add them manually.

Listing 14.8 CSV to a `seq` of maps

```
import akka.stream.alpakka.csv.scaladsl.CsvToMap

class CSVSpec {
  ...

  "A CSV with NO headers" should "be able to get into a Map" in {
    val result: Future[Seq[Map[String, ByteString]]] =
      Source
        .single(ByteString("Alice,Pleasence,19\nBob,Marley,30\n"))
        .via(CsvParsing.lineScanner())
        .via(CsvToMap.withHeaders("name", "surname", "age"))     ❶
        .runWith(Sink.seq)

    val lines = result.futureValue
    lines.head should be(
      Map(
        "name" -> ByteString("Alice"),
```